
Toward Faster Mamba Kernels: Performance Improvements and Hardware Limits

Batu El
Stanford University
batuel@stanford.edu

Abstract

While kernels for state-space models offer linear-time sequence processing, they pose distinct optimization challenges on AI accelerators. We study the Mamba kernel on AWS Trainium 2 (NeuronCore-v3), starting from the official NKI base-lines, which already provide a $878\times$ speedup compared to the PyTorch reference implementation, and apply four hardware-aware optimizations: (1) precomputing the state-invariant $\Delta \cdot u$ product to eliminate redundant arithmetic, (2) reordering loops to improve compute-memory overlap, (3) downcasting to BF16 to double Vector Engine throughput, and (4) increasing sequence tile sizes to reduce DMA round-trips (Section 4). Together, these optimizations yield up to a 42.5% reduction in latency over the strongest baseline while preserving Mamba’s linear scaling with sequence length (Table 1). Through profiling with Neuron Explorer, we identify the associative scan executed on the Vector Engine as the dominant bottleneck, showing that latency correlates strongly with Vector Engine utilization across all kernel variants (Figure 6). Based on this analysis, we propose three architectural modifications to enable further acceleration beyond what software optimization alone can achieve (Section 5): (1) a wider Vector Engine datapath, (2) a dedicated hardware associative scan unit, and (3) increased on-chip SBUF capacity. <https://github.com/batu-el/faster-mamba>

1 Introduction

Recently, state-space models [Gu et al., 2022, Gu and Dao, 2024, Dao and Gu, 2024, Lahoti et al., 2026, De et al., 2024, Yang et al., 2025, Sun et al., 2023, Poli et al., 2023, Fu et al., 2023, Smith et al., 2023] have emerged as a compelling alternative to attention-based Transformers [Vaswani et al., 2023] whose quadratic complexity in sequence length incurs prohibitive costs for long contexts. In this project, we focus on Mamba [Gu and Dao, 2024], which introduced the selective state-space (S6) layer that offers linear sequence processing while maintaining strong modeling performance.

1.1 Motivation

The Mamba layer is an attractive target for optimization because its performance is driven by how efficiently the system moves data and manages sequential dependencies. AWS Trainium is particularly well suited for this study for three key reasons: First, its software-managed SRAM hierarchy and parallel DMA engines allow explicit control over data movement. Second, AWS already provides a fused Mamba kernel in NKI that achieves significant speedup over the PyTorch implementation, offering a high-performance baseline. Third, Neuron Explorer allows us to analyze hardware-level performance characteristics on Trainium, which allows us to identify the hardware bottlenecks.

1.2 Contributions

Our project has two main contributions.

We further accelerate the Mamba kernel. Starting from an official AWS NKI baseline, which itself is heavily optimized and $878\times$ faster than the PyTorch reference implementation, we apply four targeted optimizations to further reduce end-to-end latency on Trainium 2 (Section 4)

Precomputing Δu : We precompute the state-invariant product $\Delta \cdot u$ outside of the inner loop, eliminating $N-1$ redundant element-wise multiplications per channel tile.

Loop reordering: We swap the state and sequence-tile loops. This improves overlap between compute and memory operations and increasing VectorE utilization.

BF16 quantization: We downcast inputs and intermediates to BF16, doubling VectorE throughput for the element-wise and scan operations that dominate execution time.

Adaptive tile sizing: We increase the sequence tile from 512 to 1024 when the sequence length permits, halving the number of outer-loop iterations.

Together, these optimizations yield up to a 42.5% reduction in latency over the already heavily optimized baseline (v3), while preserving Mamba’s linear scaling of latency with sequence length.

We identify fundamental hardware bottlenecks and propose architectural changes to enable further acceleration. Through systematic profiling, we identify the Vector Engine and the associative scan it executes as the dominant bottleneck in the kernel. We show that the latency correlates strongly with VectorE utilization across all kernel variants and sequence lengths and propose three architectural modifications that would reduce latency: (1) widening the VectorE datapath to increase parallelism, (2) adding a dedicated hardware associative scan unit to accelerate the core recurrence, and (3) increasing the on-chip SBUF size to enable larger sequence tiles and fewer DMA round-trips.

2 Preliminaries

2.1 Target ML Workload

We target the selective state-space layer (Algorithm 1), which performs a recurrent update over a sequence of length l .

$$\bar{A}_t = \text{diag}(\exp(\Delta_t A)), \quad (1)$$

$$h_t = \bar{A}_t h_{t-1} + \Delta_t B_t u_t, \quad (2)$$

$$y_t = C_t^\top h_t, \quad (3)$$

where $u_t \in \mathbb{R}$ is the input, $h_t \in \mathbb{R}^n$ is the hidden state with $h_0 = \mathbf{0}$, $y_t \in \mathbb{R}$ is the output, $\Delta_t \in \mathbb{R}$ is the discretisation step, $A \in \mathbb{R}^n$ is the static state transition parameter, $B_t \in \mathbb{R}^n$ is the input projection, and $C_t \in \mathbb{R}^n$ is the output projection. Since $\bar{A}_t \in \mathbb{R}^{n \times n}$ is diagonal, the state update reduces to an element-wise operation. This recurrence is applied independently across d channels and b batch elements, yielding a workload with shape (b, l, d, n) . In Algorithm 1, the inputs are $\text{delta} \in \mathbb{R}^{b \times d \times l}$, $u \in \mathbb{R}^{b \times d \times l}$, $A \in \mathbb{R}^{d \times n}$, $B \in \mathbb{R}^{b \times n \times l}$, and $C \in \mathbb{R}^{b \times n \times l}$, where b is the batch size, d the number of channels, n the state size, and l the sequence length. In the PyTorch implementation, all tensors are first broadcast-expanded to a common shape $\mathbb{R}^{b \times d \times n \times l}$ via None-indexing. The discretised transition matrix is computed as $\text{deltaA} = \exp(\text{delta} \odot A)$ and the input contribution as $\text{deltaB_u} = \text{delta} \odot B \odot u$, both element-wise under broadcasting.

```

1 def mamba_layer(delta, A, B, u, C):
2     deltaA = torch.exp(delta[:, :, None, :] * A[None, :, :, None])
3     deltaB_u = (
4         delta[:, :, None, :] * B[:, None, :, :] * u[:, :, None, :]
5     )
6     scan_res = associative_scan(deltaA, deltaB_u)
7     mamba_out = C[:, None, :, :] * scan_res ).sum(dim=-2)
8     return mamba_out

```

Algorithm 1: Mamba layer forward pass in PyTorch (reference implementation)

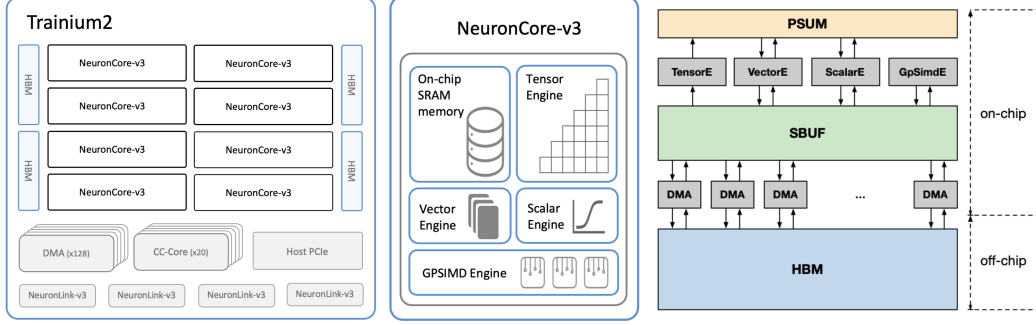


Figure 1: *Left*. Trainium2 chip architecture featuring eight NeuronCore-v3 cores, HBM stacks, DMA engines, collective compute cores, and NeuronLink-v3 interconnects. *Middle*. Internal architecture of a single NeuronCore-v3, comprising on-chip SRAM, Tensor, Vector, Scalar, and GPSIMD compute engines. *Right*. NeuronCore-v3 memory hierarchy and dataflow, showing the relationship between HBM (off-chip), SBUF, PSUM (on-chip), and the four compute engines connected via DMA. (Image sources: Trainium2 Architecture Guide for NKI, NeuronCore-v3 Architecture, Trainium2 Architecture.)

These two tensors are then passed to an associative scan (Algorithm 2), which computes the scaled prefix sum (scale by deltaA and sum with deltaB_u) over the sequence dimension, yielding the hidden states $\text{scan_res} \in \mathbb{R}^{b \times d \times n \times l}$. Finally, the output is obtained by multiplying scan_res with C and summing over the state dimension n , producing $\text{mamba_out} \in \mathbb{R}^{b \times d \times l}$.

```

1  def associative_scan(deltaA, deltaB_u):
2      batch_size, channels, state_size, seq_len = deltaA.shape
3      out = torch.empty(batch_size, channels, state_size, seq_len,
4                          device=deltaA.device, dtype=deltaA.dtype)
5      for i in range(seq_len):
6          prev_state = out[..., i - 1] if i > 0 else 0
7          out[..., i] = (deltaA[..., i] * prev_state + deltaB_u[..., i])
8      return out

```

Algorithm 2: Associative scan over the sequence dimension. (reference implementation)

2.2 Hardware

Figure 1 illustrates the Trainium 2 architecture at three levels of detail. The Trainium 2 chip integrates eight NeuronCore-v3 connected to 96 GiB of HBM via 128 DMA engines (16 per core) and with 20 collective-compute (CC) cores for inter-chip communication. We use a single logical core, which has two physical cores, for our experiments in this project. The two physical cores operate as a single unified unit with no explicit synchronization needed between them.

Compute. A single NeuronCore-v3 is a fully independent compute unit containing four specialized engines: a *Tensor Engine* (TensorE) with 128×128 systolic arrays delivering 158 cFP8 / 79 BF16 / 20 FP32 dense TFLOPS, a *Vector Engine* (VectorE) for multi-element reductions at 1 TFLOPS FP32, a *Scalar Engine* (ScalarE) for element-wise operations including non-linear functions at 1.2 TFLOPS FP32, and a general-purpose *GpSimd Engine* (GpSimdE) with eight programmable 512-bit processors for arbitrary C/C++ code (NeuronCore-v3 Architecture).

Computation in Mamba Kernel. For the Mamba kernel, ScalarE handles the fused multiply-and-exp discretization (`nisa.activation`), while VectorE performs the element-wise products and the sequential scan (`tensor_tensor_scan`).

Memory. The NeuronCore-v3 memory hierarchy has three software-managed tiers: 28 MiB of on-chip SBUF (State Buffer), 2 MiB of PSUM (Partial Sum Buffer) dedicated to TensorE accumulation, and a 96 GiB off-chip HBM (High Bandwidth Memory). The chip provides 2.9 TB/s of HBM bandwidth (shared across 8 cores), while SBUF offers approximately $20\times$ higher bandwidth than HBM, ensuring

that on-chip compute engines are not starved for data when tiles are in SBUF. The data must be loaded from HBM into SBUF via DMA before computation, and results must be written back after.

Memory Management in Mamba Kernel. In our Mamba kernels, input tiles of Δ , u , B , and C are loaded from HBM into SBUF via DMA at each loop iteration, and the loop reorderings aim to keep the largest tensors resident in SBUF across the `state_size` inner loop to maximize on-chip reuse and minimize redundant HBM transfers.

Setup. We use the private AMI Deep Learning AMI Neuron PyTorch 2.8 (Ubuntu 22.04) and a `trn2.3xlarge` instance to run our experiments. Details of our setup may be found at https://github.com/stanford-cs149/asst4-trainium2/blob/main/cloud_readme.md.

3 Baselines

We begin with the official Fused Mamba NKI kernels provided by AWS as our baselines. The kernel source code and tutorial are available. These implementations are optimized for Trainium 1 (NeuronCore-v2) and includes three versions (v1, v2, v3), each addressing specific performance bottlenecks such as data layout, compute engine mapping, and sequence-length tiling. In this section, we present the baselines and discuss their implementation.

3.1 Baseline: Initial Implementation (v1)

```

1  # Input: delta[B,D,L], u[B,D,L], A[D,N], B[B,N,L], C[B,N,L]
2  output[B, D, L] = 0
3  for b in batch: # batch loop
4      accum[D/128, 128, L] = 0
5      for n in state_size: # state loop
6          for d_tile in tiles(D, size=128): # channel loop (tiled)
7              delta_t = load delta[b, d_tile, :]
8              A_t = load A[d_tile, n]
9              deltaA = exp(delta_t * A_t)
10             u_t = load u[b, d_tile, :]
11             B_t = load B[b, n, :]
12             deltaBu = delta_t * B_t * u_t
13             x = associative_scan(deltaA, deltaBu)
14             C_t = load C[b, n, :]
15             accum[d_tile] += x * C_t
16         for d_tile in tiles(D, size=128):
17             store output[b, d_tile, :] = accum[d_tile]
```

Algorithm 3: Pseudocode for v1

Algorithm 3 demonstrates the first baseline implementation. This implementation has loop order [batch, state, channel]. In the inner-most loop, channels are tiled into groups of 128 along the partition dimension. Within each tile, ΔA is computed using a fused exponential-scale operation (`nisa.activation`), and ΔBu is formed via element-wise multiplications (`nisa.tensor_tensor`) with broadcasting of B across the channel dimension. The recurrence is then computed using a parallel associative scan (`nki.isa.tensor_tensor_scan`), and the result is multiplied by C (again broadcast across channels) and accumulated into an SBUF-resident buffer. After all states have been processed, the accumulated result is stored back to HBM tile by tile.

Limitations. This approach requires N passes over the channel tiles, reloading Δ and u from HBM for every state index. Since Δ and u are shared across all states, they can instead be loaded once and retained in SBUF, which motivates the loop reordering in v2. Additionally, v1 also maintains a 3D accumulator `accum[D/128, 128, L]` in SBUF across all iterations, since the store can only occur after all states (not necessarily all channels) have been processed. Loop reordering also addresses this by reducing the accumulator to a single tile-sized `accum[128, L]`.

3.2 Baseline: Loop Reordering (v2)

```

1  # Input: delta[B,D,L], u[B,D,L], A[D,N], B[B,N,L], C[B,N,L]
2  output[B, D, L] = 0
3  for b in batch: # batch loop
4      for d_tile in tiles(D, size=128): # channel loop (tiled)
5          accum[128, L] = 0
6          delta_t = load delta[b, d_tile, :]
7          u_t = load u[b, d_tile, :]
8          for n in state_size: # state loop
9              A_t = load A[d_tile, n]
10             deltaA = exp(delta_t * A_t)
11             B_t = load B[b, n, :]
12             deltaBu = delta_t * B_t * u_t
13             x = associative_scan(deltaA, deltaBu)
14             C_t = load C[b, n, :]
15             accum += x * C_t
16     store output[b, d_tile, :] = accum

```

Algorithm 4: Pseudocode for v2

Algorithm 4 demonstrates the second baseline implementation. This implementation has loop order [batch, channel, state]. v2 reorders the loops so that the channel tile loop is outer and the state loop is inner. This allows Δ and u to be loaded once per channel tile and reused across all state indices, eliminating the redundant HBM loads present in v1. Since each tile is fully processed before moving to the next, the accumulator is reduced from the 3D $\text{accum}[D/128, 128, L]$ in v1 to a single 2D tile-sized $\text{accum}[128, L]$, and the store is fused directly after the state loop, removing the need for a separate store loop. The inner loop over states still loads A , B , and C per iteration, but these are smaller tensors: A is a single column vector and B , C are single rows. Overall, v2 reduces HBM traffic by a factor of roughly N for Δ and u (previously, Δ and u were loaded again for each iteration of the outer state loop, which is N times) while keeping the number of compute operations the same.

Limitations. While this version eliminates redundant loads of Δ and u , the kernel does not tile the sequence length dimension. As L grows, the active tile sizes of input and intermediate tensors grow proportionally in the free dimension, causing spill/reload traffic. v3 addresses this by introducing a tiling loop over the sequence dimension.

3.3 Baseline: Sequence Length Tiling (v3)

```

1  # Input: delta[B,D,L], u[B,D,L], A[D,N], B[B,N,L], C[B,N,L]
2  output[B, D, L] = 0
3  for b in batch: # batch loop
4      for d_tile in tiles(D, size=128): # channel loop (tiled)
5          accum[128, L] = 0
6          delta_t = load delta[b, d_tile, :]
7          u_t = load u[b, d_tile, :]
8          for n in state_size: # state loop
9              A_t = load A[d_tile, n]
10             scan_init[128, 1] = 0
11             for l_tile in tiles(L, size=512): # sequence length loop (tiled)
12                 deltaA = exp(delta_t[d_tile, l_tile] * A_t)
13                 B_t = load B[b, n, l_tile]
14                 deltaBu = delta_t[d_tile, l_tile] * B_t * u_t[d_tile, l_tile]
15                 x = associative_scan(deltaA, deltaBu, initial=scan_init)
16                 scan_init = x[:, -1]
17                 C_t = load C[b, n, l_tile]
18                 accum[d_tile, l_tile] += x * C_t
19     store output[b, d_tile, :] = accum

```

Algorithm 5: Pseudocode for v3

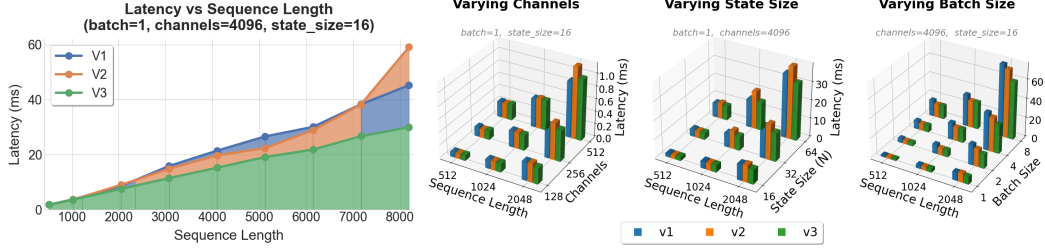


Figure 2: *Left.* Latency comparison of v1, v2, and v3 as a function of sequence length with batch size 1, 4096 channels, and state size 16. v3 consistently achieves the lowest latency across all sequence lengths, while v2 exhibits the highest latency at longer sequences. *Right.* Latency breakdown when independently varying the number of channels (with batch=1, state_size=16), state size (with batch=1, channels=4096), and batch size (with channels=4096, state_size=16) across multiple sequence lengths. v3 maintains competitive or superior performance across all parameter configurations.

Algorithm 5 introduces an additional tiling loop over the sequence dimension, partitioning L into chunks of 512. This allows intermediate tensors to fit within a smaller working set per iteration and mitigates the spills from SBUF that arises as L grows. Because the computation for position $i + 1$ depends on the result of position i in associative scan, the sequence tiles must be processed in order. Therefore, the last column of each scan result becomes the `scan_init` for the next tile. Loads of B and C are now scoped to individual sequence tiles rather than spanning the full sequence length, while Δ and u are still loaded once per channel tile and indexed into per sequence tile. The overall loop structure and HBM traffic for Δ and u remain unchanged from v2, but the reduced per-iteration memory footprint improves SBUF utilization and enables the kernel to scale to longer sequences without degradation.

3.4 Comparison between Baselines

As shown in Figure 2, we compare the baseline latencies of v1, v2, and v3 across varying sequence lengths, batch sizes, channel counts, and state sizes. Overall, v3 consistently achieves significantly lower latencies across all configurations. Therefore, we compare our optimized kernels to v3, the strongest baseline. Notably, v3 is reported to be 878x faster compared to a PyTorch reference implementation. With our optimizations, we further accelerate this already very strong baseline.

4 Optimizations and Analysis

In this section, we describe further optimizations to the baseline implementation. Starting from v3, the fastest baseline kernel, we apply four targeted modifications whose individual contributions are shown in Table 1 (rows b1–b4). Together, these changes yield an additional $\sim 40\%$ reduction in latency. The following subsections detail each optimization.

Seq. Len.	Baselines (ms)			Optimized Kernels (ms)				Improvement over v3 (%)			
	v1	v2	v3	b1	b2	b3	b4	b1	b2	b3	b4
512	1.86	1.79	1.78	1.78	1.64	1.17	1.17	0.0	7.9	34.3	34.3
1024	3.73	3.63	3.60	3.55	3.27	2.33	2.16	1.4	9.2	35.3	40.0
2048	8.59	9.00	7.54	7.30	6.67	4.67	4.34	3.2	11.5	38.1	42.4
4096	21.46	19.76	15.23	15.13	13.82	9.86	8.75	0.7	9.3	35.3	42.5
8192	45.23	59.09	29.97	34.64	28.04	20.32	17.93	-15.6	6.4	32.2	40.2

Table 1: *Performance of Baselines and Optimized Kernels.* Reported latency numbers are the median of 100 runs measured by calling `nki.benchmark()`. Improvement over v3 is computed as percentage reduction in latency, $(v3 - b)/v3 \times 100$, where positive values indicate lower latency. We use improvement over v3, because it is the strongest baseline implementation.

4.1 Precomputing Δu (b1)

In Algorithm 5, lines 6 and 7 load `delta_t` and `u_t`, yet the only subsequent use of `u_t` within the loop is in the computation of `deltaBu`, where it is multiplied by `delta_t`. Rather than performing this multiplication repeatedly inside the loop, we propose precomputing this product immediately after loading both values.

```

1  # Input: delta[B,D,L], u[B,D,L], A[D,N], B[B,N,L], C[B,N,L]
2  output[B, D, L] = 0
3  for b in batch:
4      for d_tile in tiles(D, size=128):
5          accum[128, L] = 0
6          delta_t = load delta[b, d_tile, :]
7          u_t = load u[b, d_tile, :]
8          deltaU = delta_t * u_t # precomputed once
9          for n in state_size:
10             A_t = load A[d_tile, n]
11             scan_init[128, 1] = 0
12             for l_tile in tiles(L, size=512):
13                 deltaA = exp(delta_t[d_tile, l_tile] * A_t)
14                 B_t = load B[b, n, l_tile]
15                 deltaBu = deltaU[d_tile, l_tile] * B_t # reuse precomputed
16                 x = associative_scan(deltaA, deltaBu, initial=scan_init)
17                 scan_init = x[:, -1]
18                 C_t = load C[b, n, l_tile]
19                 accum[d_tile, l_tile] += x * C_t
20             store output[b, d_tile, :] = accum

```

Algorithm 6: Pseudocode for the optimized version that precomputes `deltaU` (b1)

b1 extends v3 by precomputing the $\Delta \cdot u$ product out of the state loop. In v3, the three-way product $\Delta \cdot B \cdot u$ is recomputed inside the inner state loop for every state index, even though $\Delta \cdot u$ is state-invariant. b1 instead computes `deltaU` once on the full $(128, L)$ tile immediately after loading Δ and u , and the inner loop then forms $\Delta B u$ as `deltaU` $\cdot B$.

Analysis. This is an algorithmic improvement that saves $N-1$ element-wise multiplications per channel tile. At first glance, it appears that we increase the SBUF footprint since we now store an extra tensor `deltaU`; however, note that `u_t` is no longer needed in the SBUF after `deltaU` has been computed, so the footprint remains unchanged. As shown in Table 1, b1 achieves better performance than v3 across all sequence lengths except 8k. To understand why, we turn our attention to the profiling results (Table 2).

Metric	v1	v2	v3	b1	b2	b3	b4
Total (ms)	45.254	59.114	29.987	34.656	28.057	20.341	17.935
Tensor %	2.000	48.100	94.600	84.100	3.800	1.300	1.400
Vector %	59.100	46.900	83.100	71.800	88.300	89.200	94.400
Scalar %	9.100	14.800	21.700	18.400	25.400	34.800	28.900
DMA %	47.800	21.800	11.200	9.500	12.800	41.500	13.000
Active %	97.200	97.600	99.800	99.800	95.900	96.500	97.500
HBM Read (MB)	6309.500	3958.900	301.400	301.400	536.700	1733.000	405.200
HBM Write (MB)	2147.500	234.900	134.200	134.200	266.300	471.100	201.300
MBU %	26.100	9.910	2.030	1.760	4.000	15.130	4.720
HW FLOPS (M)	134.200	4295.000	4295.000	4295.000	134.200	67.100	67.100
Arith Intensity	0.020	1.020	9.860	9.860	0.170	0.030	0.110

Table 2: *Hardware profiling results for all kernel variants at sequence length $L=8192$.* Utilization percentages for the Tensor, Vector, Scalar, and DMA engines indicate each unit’s share of active cycles. MBU denotes memory bandwidth utilization, and arithmetic intensity is measured in FLOPS per byte transferred.

In Table 2, when comparing the columns for v3 and b1, we observe that HBM traffic, hardware FLOPS, and arithmetic intensity are identical, but the Tensor engine utilization drops from 94.6% to

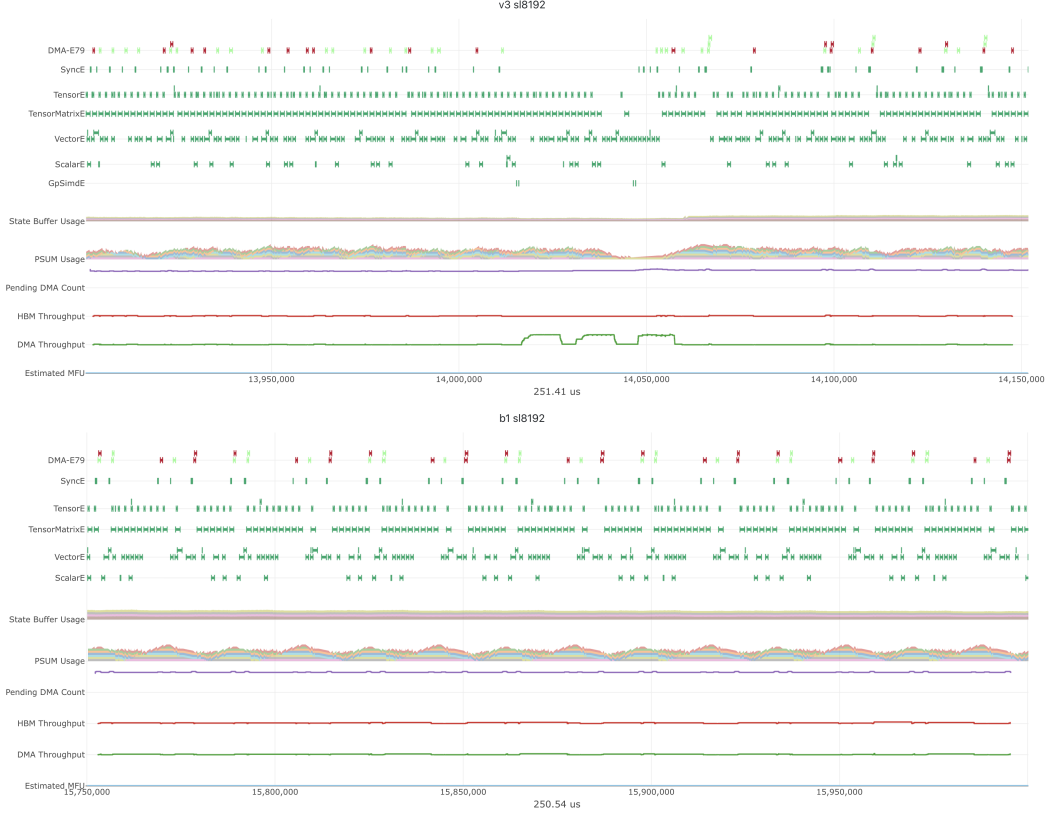


Figure 3: *Neuron Profile execution traces for v3 (top) and b1 (bottom) at $L=8192$. Each row corresponds to a hardware engine (DMA, Sync, TensorE, TensorMatrixE, VectorE, ScalarE), followed by State Buffer and PSUM usage, pending DMA count, HBM and DMA throughput, and estimated MFU. Both slices span approximately 250 μ s.*

84.1% and Vector utilization drops from 83.1% to 71.8%. This suggests the diminished performance of b1 at $L=8192$ may be due to a pipelining issue. In v3 the $\text{delta} * u$ multiplication is interleaved with DMA loads and the associative scan inside the inner loop, allowing it to overlap with memory operations, whereas b1 precomputes this work into a large upfront compute phase over the full $(128, L)$ tile. At long sequence lengths this front-loaded block is substantial, and the inner loop loses the overlapping arithmetic that previously helped hide latencies. To confirm this hypothesis, we take a closer look at the profiling results in Figure 3. The execution traces corroborate this interpretation: the TensorE and TensorMatrixE rows in v3 exhibit dense, nearly continuous activity, whereas b1 shows more fragmented bursts with frequent small gaps, limiting overlap opportunities.

4.2 Loop Reordering (b2)

To improve pipelining, in b2, we swap b1’s loop order from $[\text{batch}, \text{channel_tile}, \text{state}, \text{seq_tile}]$ to $[\text{batch}, \text{channel_tile}, \text{seq_tile}, \text{state}]$, moving sequence length tiling outside the state loop. As a result, Δ , u , and the precomputed $\text{delta}U$ are loaded and computed as small $(128, 512)$ tiles rather than full $(128, L)$ tensors, dramatically reducing the SBUF footprint from three or four full-length buffers down to tile-sized ones. The accumulator is similarly reduced to $(128, 512)$ and stored immediately after each sequence tile completes, freeing SBUF for the next tile. To maintain correct recurrence across sequence tile boundaries, per-state scan carries $\text{scan_inits}[N]$ are persisted across the outer sequence loop. The trade-off is that A , B , and C must now be reloaded for every sequence tile rather than once per state iteration over the full sequence. This is reflected in Table 2, where b2’s HBM reads increase from 301.4 MB to 536.7 MB compared to b1, yet total latency improves because VectorE engine utilization, which is carrying out the scan operation, increases to 88.3%. VectorE row now exhibits a dense, regular pattern of



Figure 4: *Neuron Profile* execution traces for $b2$ at $L=8192$. The top panel spans approximately $250\mu s$ and shows VectorE, ScalarE, SyncE, and GpSimdE activity alongside State Buffer usage, pending DMA count, and DMA throughput. The bottom panel zooms into a $\sim 50\mu s$ window, revealing the regular, repeating pattern of vector and scalar operations within each sequence tile.

operations within each sequence tile. *Since the scan operation appears to be the main bottleneck, we target increasing VectorE utilization to reduce latency.*

```

1 # Input: delta[B,D,L], u[B,D,L], A[D,N], B[B,N,L], C[B,N,L]
2 output[B, D, L] = 0
3 for b in batch:
4     for d_tile in tiles(D, size=128):
5         scan_inits[N][128, 1] = 0
6         for l_tile in tiles(L, size=512):
7             delta_t = load delta[b, d_tile, l_tile]
8             u_t = load u[b, d_tile, l_tile]
9             deltaU = delta_t * u_t
10            accum[128, 512] = 0
11            for n in state_size:
12                A_t = load A[d_tile, n]
13                deltaA = exp(delta_t * A_t)
14                B_t = load B[b, n, l_tile]
15                deltaBu = deltaU * B_t
16                x = associative_scan(deltaA, deltaBu, initial=scan_inits[n])
17                scan_inits[n] = x[:, -1]
18                C_t = load C[b, n, l_tile]
19                accum += x * C_t
20            store output[b, d_tile, l_tile] = accum

```

Algorithm 7: Pseudocode for optimized version that reorders the loops ($b2$)

4.3 Quantization (b3)

```

1 # Input: delta[B,D,L], u[B,D,L], A[D,N], B[B,N,L], C[B,N,L]
2 output[B, D, L] = 0
3 for b in batch:
4     for d_tile in tiles(D, size=128):
5         scan_inits[N][128, 1] = 0
6         for l_tile in tiles(L, size=512):
7             delta_t = load delta[b, d_tile, l_tile] as bf16
8             u_t = load u[b, d_tile, l_tile] as bf16
9             deltaU = delta_t * u_t
10            accum[128, 512] = 0
11            for n in state_size:
12                A_t = load A[d_tile, n]
13                deltaA = exp(delta_t * A_t)
14                B_t = load B[b, n, l_tile] as bf16
15                deltaBu = deltaU * B_t
16                x = associative_scan(deltaA, deltaBu, initial=scan_inits[n])
17                scan_inits[n] = x[:, -1]
18                C_t = load C[b, n, l_tile] as bf16
19                accum += x * C_t
20            store output[b, d_tile, l_tile] = accum

```

Algorithm 8: Pseudocode for optimized version that implements quantization (b3)

Having identified VectorE as the primary bottleneck in b2, b3 targets its throughput directly by downcasting computation to BF16. All loads of Δ , u , B , and C are cast to BF16, and the accumulator and scan carries are likewise BF16. On NeuronCore-v3, this triggers the VectorE performance mode, providing $2\times$ throughput for element-wise operations (`nisa.tensor_tensor`). BF16 also halves the per-element memory footprint in SBUF, further reducing pressure on the already tile-sized working set. The loop order, precomputed ΔU , per-state scan carries, and immediate per-tile store remain identical to b2. Table 1 demonstrated that quantization results in a significant performance boost across all sequence lengths.¹ Figure 5 (top) confirms that VectorE remains the dominant engine.

4.4 Tile Sizes (b4)

```

1 # Input: delta[B,D,L], u[B,D,L], A[D,N], B[B,N,L], C[B,N,L]
2 output[B, D, L] = 0
3 S = 1024 if L >= 1024 else 512
4 for b in batch:
5     for d_tile in tiles(D, size=128):
6         scan_inits[N][128, 1] = 0
7         for l_tile in tiles(L, size=S):
8             delta_t = load delta[b, d_tile, l_tile] as bf16
9             u_t = load u[b, d_tile, l_tile] as bf16
10            deltaU = delta_t * u_t
11            accum[128, S] = 0
12            for n in state_size:
13                A_t = load A[d_tile, n]
14                deltaA = exp(delta_t * A_t)
15                B_t = load B[b, n, l_tile] as bf16
16                deltaBu = deltaU * B_t
17                x = associative_scan(deltaA, deltaBu, initial=scan_inits[n])
18                scan_inits[n] = x[:, -1]
19                C_t = load C[b, n, l_tile] as bf16
20                accum += x * C_t
21            store output[b, d_tile, l_tile] = accum

```

Algorithm 9: Pseudocode for optimized version that with tuned tile sizes (b4)

¹We note that quantization leads to reduced precision, which requires us to adjust the tolerance levels for our accuracy checks.



Figure 5: *Neuron Profile* execution traces for *b3* (top) and *b4* (bottom) at $L=8192$. Both panels span approximately $50\mu s$ and show SyncE, VectorE, ScalarE, and GpSimdE activity alongside State Buffer usage, pending DMA count, and DMA throughput. In *b3*, DMA throughput is visibly elevated throughout the window, consistent with its high DMA utilization (41.5%) and increased HBM traffic reported in Table 2. In *b4*, DMA activity is concentrated into a brief burst near the end of the window, reflecting the reduced HBM reads (405.2 MB vs. 1733.0 MB) achieved by restoring the outer state loop. Both traces show dense VectorE utilization, with *b4* exhibiting slightly longer continuous instruction spans and fewer idle gaps.

b4 increases *b3*'s sequence tile size from a fixed 512 to an adaptive 1024 when the sequence length permits ($L \geq 1024$), falling back to 512 for shorter sequences. This larger tile is feasible because *b3*'s BF16 quantization already halves the per-element SBUF footprint, leaving sufficient headroom for the wider tile without spilling. Doubling the tile size halves the number of outer-loop iterations and, consequently, the number of scan launches and DMA round-trips for reloading *A*, *B*, and *C*. As shown in Table 1, *b4* delivers consistent gains over *b3* at sequence lengths ≥ 1024 , reaching a **42.5% improvement over v3 at $L=4096$** ; at $L=512$ the tile size is unchanged, so performance matches *b3*. The effect is visible in Table 2: HBM reads drop from 1733.0 MB (*b3*) to 405.2 MB (*b4*), and VectorE utilization rises from 89.2% to 94.4%, while DMA utilization falls from 41.5% to 13.0%. Figure 5 corroborates this shift: in *b3* (top), DMA throughput is elevated throughout the window and VectorE activity is interspersed with frequent short gaps, whereas in *b4* (bottom) DMA activity is concentrated into a brief burst near the end of the window and VectorE exhibits longer, more continuous instruction spans.

4.5 A Note on the Main Bottleneck

As shown in Figure 6, latency correlates strongly with vector engine utilization across all sequence lengths, confirming that the associative scan on VectorE is the primary bottleneck. A linear trendline fit to the 8k sequence length data suggests that full vector engine utilization could yield a theoretical latency of 14.4ms, though this limit may not be practically achievable on current hardware.

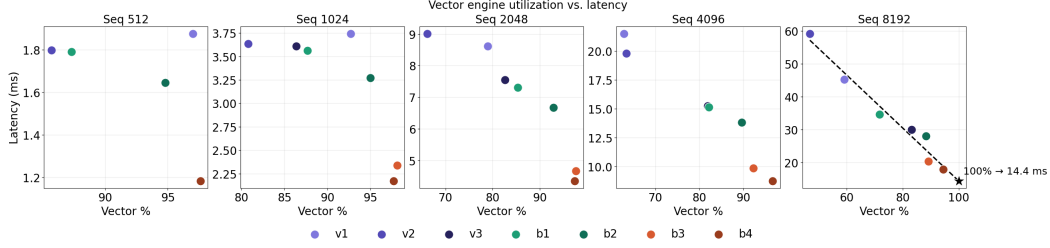


Figure 6: *Vector engine utilization vs. latency across sequence lengths.* Each point represents a kernel variant (v1–v3, b1–b4) evaluated at a given sequence length. Across all sequence lengths, higher vector engine utilization consistently correlates with lower latency. The linear trend becomes clearer as sequence length increases.

5 Hardware Modifications for Further Acceleration

In this section, we characterize the remaining bottlenecks and identify which hardware constraints ultimately limit further acceleration. In our best kernel (b4), VectorE utilization reaches 94.4%, ScalarE reaches 28.9%, and DMA utilization sits at 13.0%, with an SBUF capacity constraining tile dimensions. We identify the following three hardware changes that would accelerate execution.

5.1 Proposal 1. Wider Vector Engine Datapath

Description. With VectorE at 94.4% utilization in b4, the engine is near saturation and further software optimization yields diminishing returns. We propose doubling the VectorE datapath width to process $2\times$ the BF16 elements per cycle, directly halving the cycle count for the element-wise operations and associative scan that dominate the inner loop.

Forecast. In b4, the scalar engine (28.9% utilization) and DMA engine (13.0%) run partly in parallel with VectorE, hence, their latencies are hidden behind the longer vector work. Doubling VectorE throughput halves the vector execution time, but scalar and DMA durations remain unchanged. Latencies that were previously overlapped with vector execution may now be exposed. This can result in idle time. Therefore, the realized speedup is less than the theoretical $2\times$. We conservatively estimate that a wider vector engine datapath would lead to a $1.4\text{--}1.7\times$ speedup at $L=8192$, which would bring b4’s 17.9 ms to approximately 10.5–12.8 ms.

5.2 Proposal 2. Dedicated Hardware Associative Scan Unit

Description. The associative scan is the dominant operation on VectorE (94.4% utilization in b4). We propose a dedicated hardware scan unit that accelerates this operation, accepting `deltaA` and `deltaBu` tiles of shape $(128, S)$ alongside an initial carry vector.

Forecasts. If a hardware scan unit with greater parallelism and pipelining reduces scan latency by $P\%$, and the scan constitutes roughly 40–60% of total execution time in b4, the resulting reduction in total latency would be approximately $0.4P\text{--}0.6P\%$. For example, even a 50% reduction in scan latency would reduce total latency by $\sim 20\text{--}30\%$, bringing the $L=8192$ configuration from 17.9 ms to approximately 12.5–14.3 ms. The kernel would then shift to being scalar or DMA-bound.

5.3 Proposal 3. Increased State Buffer (SBUF) Capacity

Description. NeuronCore-v3’s SBUF constrains the sequence tile to 1024 in b4 and further increases cause spilling. We propose doubling SBUF to enable sequence tiles of 2048 or larger.

Forecasts. Doubling the tile from 1024 to 2048 halves the outer-loop iterations, reducing DMA round-trips for B and C , scan-carry transfers, and scalar overhead. Based on the b3 to b4 trend, where doubling the tile from 512 to 1024 reduced latency by 12–15% at $L \geq 2048$, we estimate doubling the SBUF would lead to a further 10–15% reduction in latency, bringing b4 from 17.9 ms to $\sim 15.2\text{--}16.1$ ms at $L=8192$.

6 Conclusion

We optimized the Mamba kernel on AWS Trainium 2 by implementing four hardware-aware revisions (precomputation, loop reordering, BF16 quantization, and tuned tiling) achieving up to 42.5% lower latency while preserving linear scaling. Through profiling, we identified Vector Engine utilization as the primary bottleneck. Based on this analysis, we proposed three architectural improvements: a wider datapath, a dedicated associative scan unit, and increased on-chip memory. Our results suggest that as sequence lengths continue to grow, realizing the full potential of linear-time state-space models will require co-designing kernels with application-aware hardware extensions, particularly for the sequential scan primitive that, even after aggressive software optimization, remains the dominant bottleneck at over 94% Vector Engine utilization.

AI Assistance

This project used language models to assist with text revision, code implementation, and debugging. All AI-generated content was reviewed and validated by the authors.

References

- Tri Dao and Albert Gu. Transformers are ssms: Generalized models and efficient algorithms through structured state space duality, 2024. URL <https://arxiv.org/abs/2405.21060>.
- Soham De, Samuel L. Smith, Anushan Fernando, Aleksandar Botev, George Cristian-Muraru, Albert Gu, Ruba Haroun, Leonard Berrada, Yutian Chen, Srivatsan Srinivasan, Guillaume Desjardins, Arnaud Doucet, David Budden, Yee Whye Teh, Razvan Pascanu, Nando De Freitas, and Caglar Gulcehre. Griffin: Mixing gated linear recurrences with local attention for efficient language models, 2024. URL <https://arxiv.org/abs/2402.19427>.
- Daniel Y. Fu, Tri Dao, Khaled K. Saab, Armin W. Thomas, Atri Rudra, and Christopher Ré. Hungry hungry hippos: Towards language modeling with state space models, 2023. URL <https://arxiv.org/abs/2212.14052>.
- Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces, 2024. URL <https://arxiv.org/abs/2312.00752>.
- Albert Gu, Karan Goel, and Christopher Ré. Efficiently modeling long sequences with structured state spaces, 2022. URL <https://arxiv.org/abs/2111.00396>.
- Aakash Lahoti, Kevin Y. Li, Berlin Chen, Caitlin Wang, Aviv Bick, J. Zico Kolter, Tri Dao, and Albert Gu. Mamba-3: Improved sequence modeling using state space principles, 2026. URL <https://arxiv.org/abs/2603.15569>.
- Michael Poli, Stefano Massaroli, Eric Nguyen, Daniel Y. Fu, Tri Dao, Stephen Baccus, Yoshua Bengio, Stefano Ermon, and Christopher Ré. Hyena hierarchy: Towards larger convolutional language models, 2023. URL <https://arxiv.org/abs/2302.10866>.
- Jimmy T. H. Smith, Andrew Warrington, and Scott W. Linderman. Simplified state space layers for sequence modeling, 2023. URL <https://arxiv.org/abs/2208.04933>.
- Yutao Sun, Li Dong, Shaohan Huang, Shuming Ma, Yuqing Xia, Jilong Xue, Jianyong Wang, and Furu Wei. Retentive network: A successor to transformer for large language models, 2023. URL <https://arxiv.org/abs/2307.08621>.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need, 2023. URL <https://arxiv.org/abs/1706.03762>.
- Songlin Yang, Jan Kautz, and Ali Hatamizadeh. Gated delta networks: Improving mamba2 with delta rule, 2025. URL <https://arxiv.org/abs/2412.06464>.

A Supplemental Tables

Table 3: Kernel Profile — Sequence Length 512

Metric	v1	v2	v3	b1	b2	b3	b4
Total (ms)	1.875	1.798	1.790	1.790	1.645	1.183	1.184
Tensor %	3.700	98.400	98.400	98.400	3.900	1.600	1.600
Vector %	97.000	85.700	87.300	87.300	94.800	97.600	97.600
Scalar %	22.300	22.500	22.500	22.500	25.200	34.500	34.500
DMA %	52.200	5.600	5.700	5.700	5.600	6.100	6.200
Active %	99.800	100.000	99.900	99.900	99.300	99.400	99.500
HBM Read (MB)	264.000	17.100	17.100	17.100	17.100	17.100	17.100
HBM Write (MB)	8.400	8.400	8.400	8.400	8.400	8.400	8.400
MBU %	20.300	1.980	1.990	1.990	2.160	3.010	3.010
HW FLOPS (M)	8.400	268.400	268.400	268.400	8.400	4.200	4.200
Arith Intensity	0.030	10.530	10.530	10.530	0.330	0.160	0.160

Table 4: Kernel Profile — Sequence Length 1024

Metric	v1	v2	v3	b1	b2	b3	b4
Total (ms)	3.742	3.635	3.610	3.561	3.271	2.337	2.171
Tensor %	3.300	97.500	98.100	98.600	3.600	1.400	1.500
Vector %	92.700	80.800	86.400	87.700	95.000	98.100	97.700
Scalar %	17.900	45.800	22.200	22.400	25.800	35.200	29.700
DMA %	54.100	4.900	4.900	5.200	10.600	10.900	7.400
Active %	99.100	99.900	99.900	99.900	99.300	99.700	99.400
HBM Read (MB)	553.500	33.900	33.900	33.900	60.200	46.400	33.900
HBM Write (MB)	32.500	16.800	16.800	16.800	23.900	23.100	16.800
MBU %	21.870	1.950	1.960	1.990	3.590	4.150	3.260
HW FLOPS (M)	16.800	536.900	536.900	536.900	16.800	8.400	8.400
Arith Intensity	0.030	10.580	10.580	10.580	0.200	0.120	0.170

Table 5: Kernel Profile — Sequence Length 2048

Metric	v1	v2	v3	b1	b2	b3	b4
Total (ms)	8.612	9.007	7.555	7.311	6.675	4.672	4.350
Tensor %	2.700	79.200	94.600	97.000	3.800	1.300	1.300
Vector %	79.000	66.100	82.600	85.300	92.900	97.700	97.400
Scalar %	14.400	35.000	21.500	22.400	25.900	35.500	30.000
DMA %	60.800	6.800	9.200	9.800	13.600	18.300	11.700
Active %	98.700	99.300	99.600	99.900	98.600	99.800	99.400
HBM Read (MB)	1437.100	72.500	72.800	73.100	131.200	156.200	95.200
HBM Write (MB)	379.600	33.600	33.600	33.600	65.000	52.300	39.600
MBU %	29.460	1.640	1.970	2.040	4.110	6.230	4.330
HW FLOPS (M)	33.600	1073.700	1073.700	1073.700	33.600	16.800	16.800
Arith Intensity	0.020	10.130	10.090	10.070	0.170	0.080	0.120

Table 6: Kernel Profile — Sequence Length 4096

Metric	v1	v2	v3	b1	b2	b3	b4
Total (ms)	21.486	19.779	15.246	15.148	13.824	9.877	8.756
Tensor %	2.100	71.800	94.000	94.400	3.900	1.300	1.400
Vector %	62.800	63.400	81.900	82.200	89.600	92.300	96.700
Scalar %	10.300	27.800	21.500	21.700	25.500	35.000	29.400
DMA %	53.300	7.400	11.000	11.200	13.600	46.300	13.100
Active %	96.800	99.300	99.600	99.700	96.800	98.800	98.900
HBM Read (MB)	3102.500	149.700	149.300	149.300	266.600	900.600	202.500
HBM Write (MB)	979.400	67.100	67.100	67.100	132.100	270.700	100.700
MBU %	26.530	1.530	1.980	2.000	4.030	16.560	4.840
HW FLOPS (M)	67.100	2147.500	2147.500	2147.500	67.100	33.600	33.600
Arith Intensity	0.020	9.900	9.920	9.920	0.170	0.030	0.110

Table 7: Kernel Profile — Sequence Length 8192

Metric	v1	v2	v3	b1	b2	b3	b4
Total (ms)	45.254	59.114	29.987	34.656	28.057	20.341	17.935
Tensor %	2.000	48.100	94.600	84.100	3.800	1.300	1.400
Vector %	59.100	46.900	83.100	71.800	88.300	89.200	94.400
Scalar %	9.100	14.800	21.700	18.400	25.400	34.800	28.900
DMA %	47.800	21.800	11.200	9.500	12.800	41.500	13.000
Active %	97.200	97.600	99.800	99.800	95.900	96.500	97.500
HBM Read (MB)	6309.500	3958.900	301.400	301.400	536.700	1733.000	405.200
HBM Write (MB)	2147.500	234.900	134.200	134.200	266.300	471.100	201.300
MBU %	26.100	9.910	2.030	1.760	4.000	15.130	4.720
HW FLOPS (M)	134.200	4295.000	4295.000	4295.000	134.200	67.100	67.100
Arith Intensity	0.020	1.020	9.860	9.860	0.170	0.030	0.110

B Supplemental Profiles

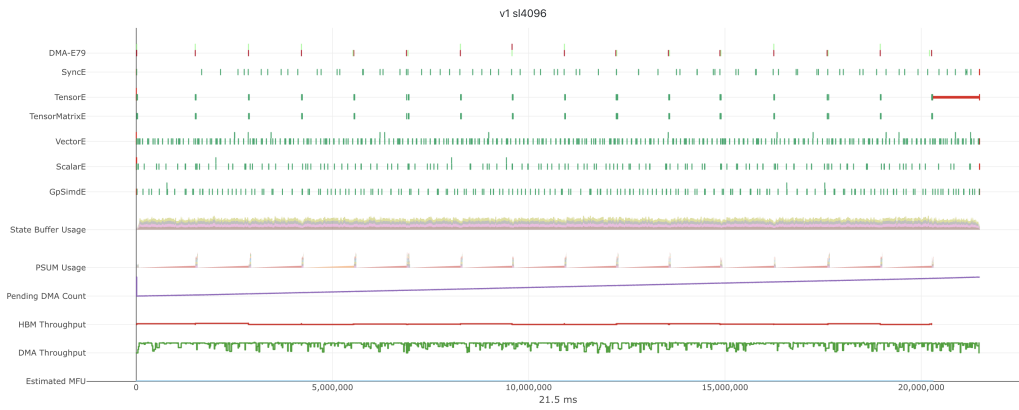


Figure 7: v1, 4k sequence length

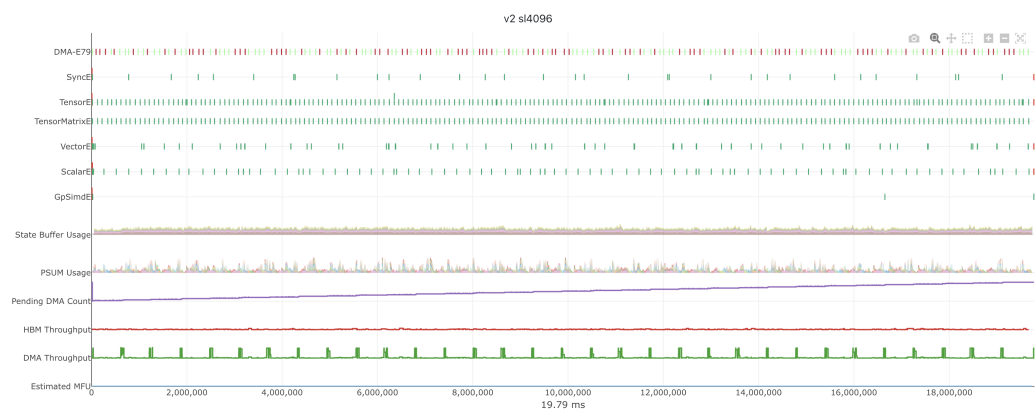


Figure 8: v2, 4k sequence length

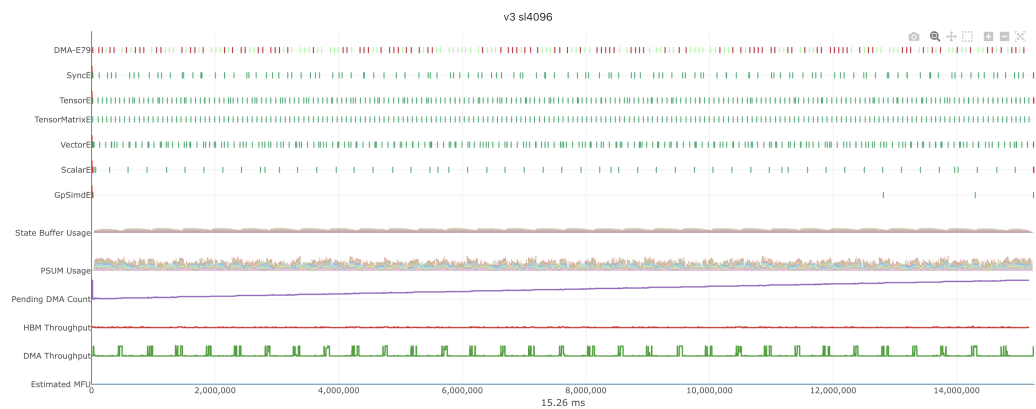


Figure 9: v3, 4k sequence length



Figure 10: b1, 4k sequence length

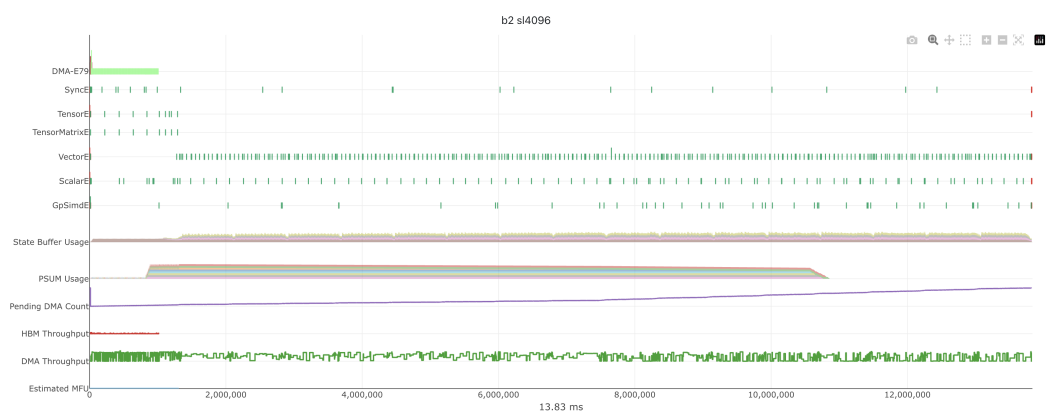


Figure 11: b2, 4k sequence length

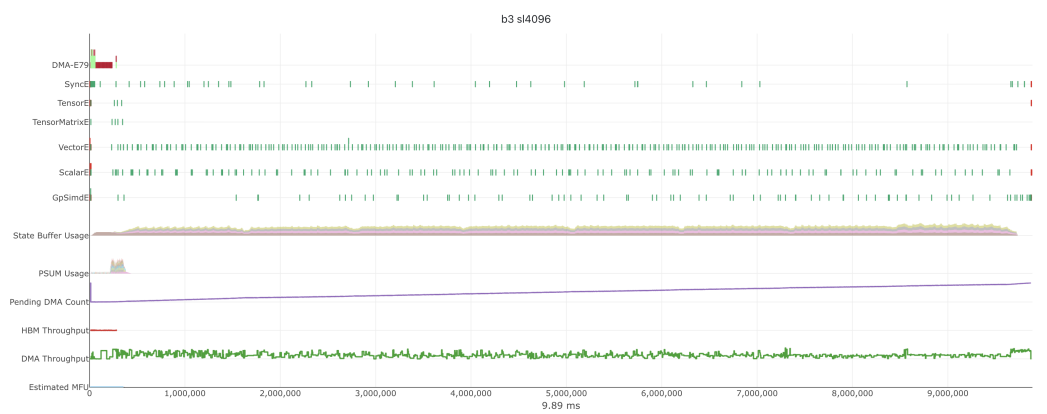


Figure 12: b3, 4k sequence length

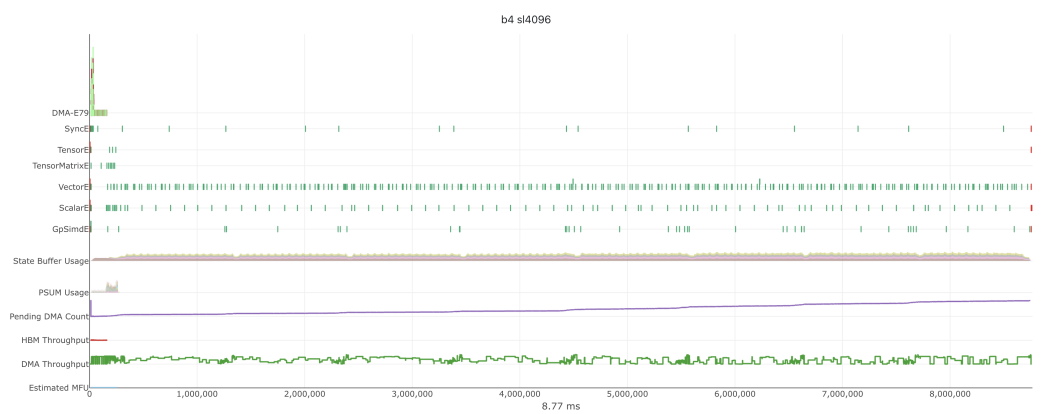


Figure 13: b4, 4k sequence length

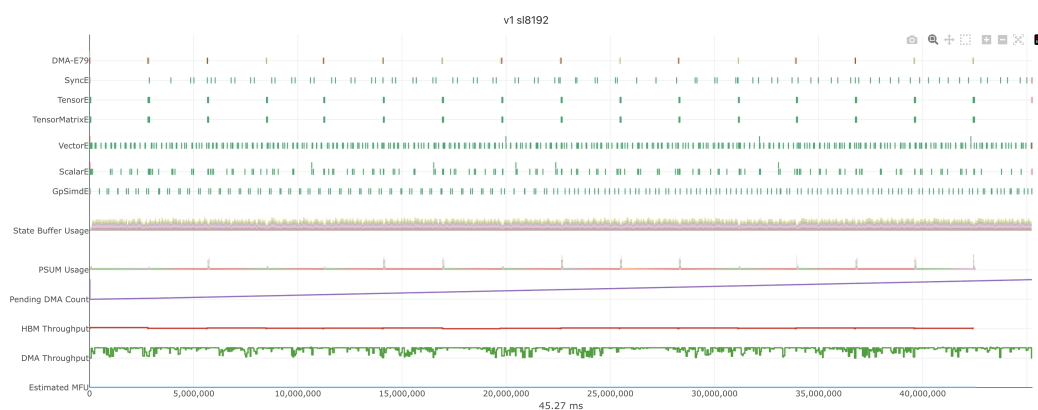


Figure 14: v1, 8k sequence length

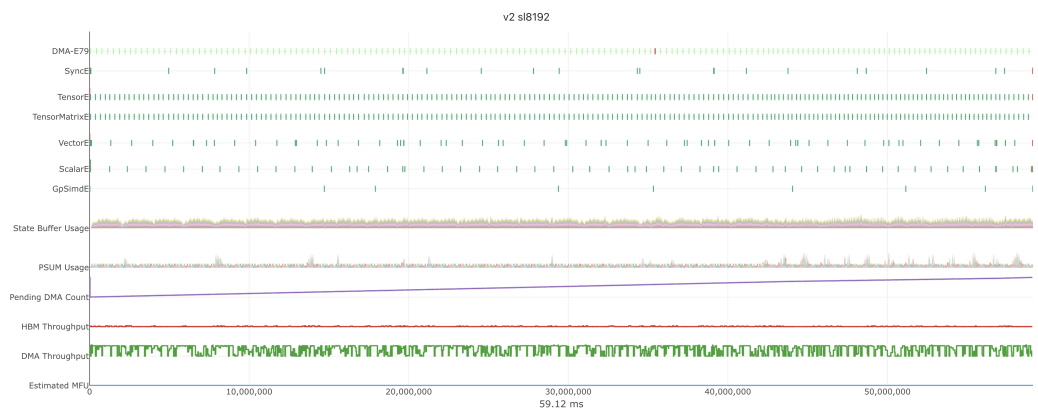


Figure 15: v2, 8k sequence length

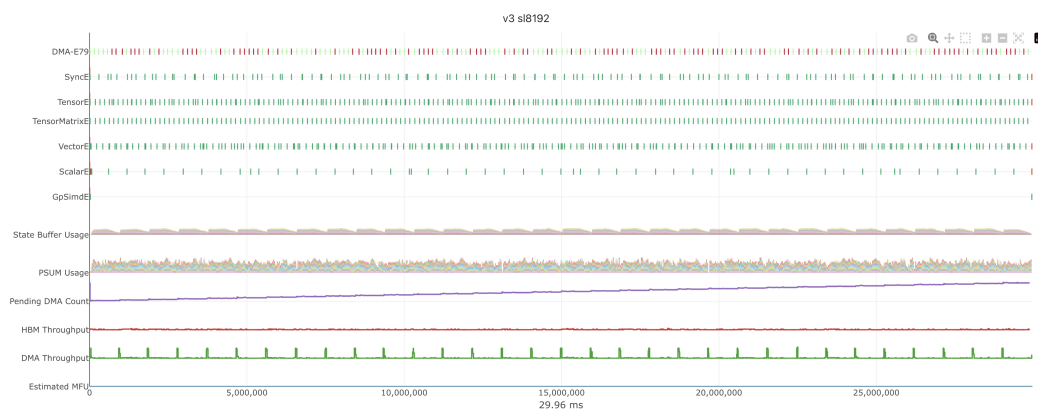


Figure 16: v3, 8k sequence length

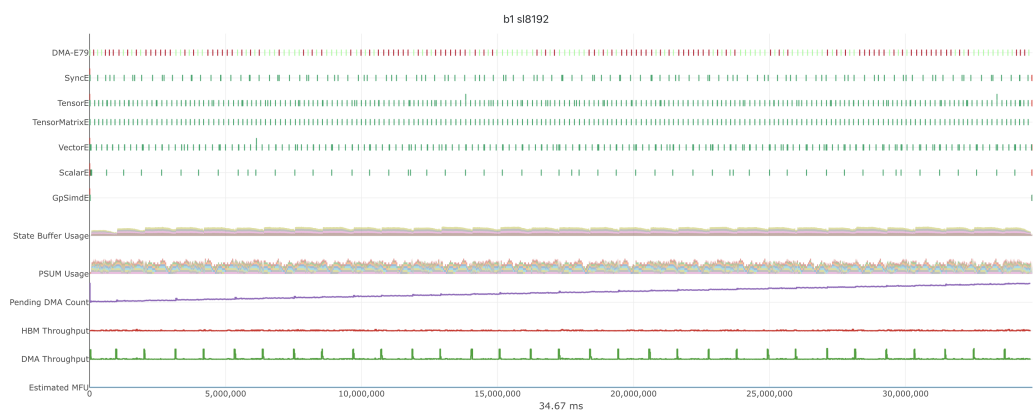


Figure 17: b1, 8k sequence length

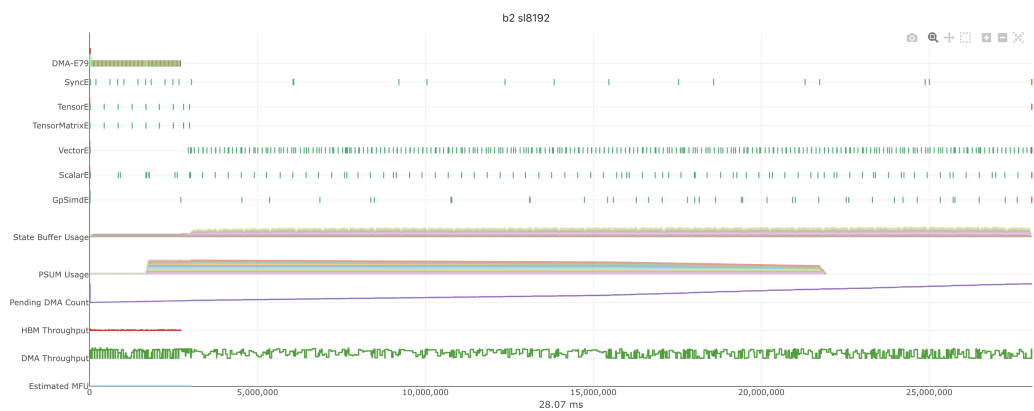


Figure 18: b2, 8k sequence length

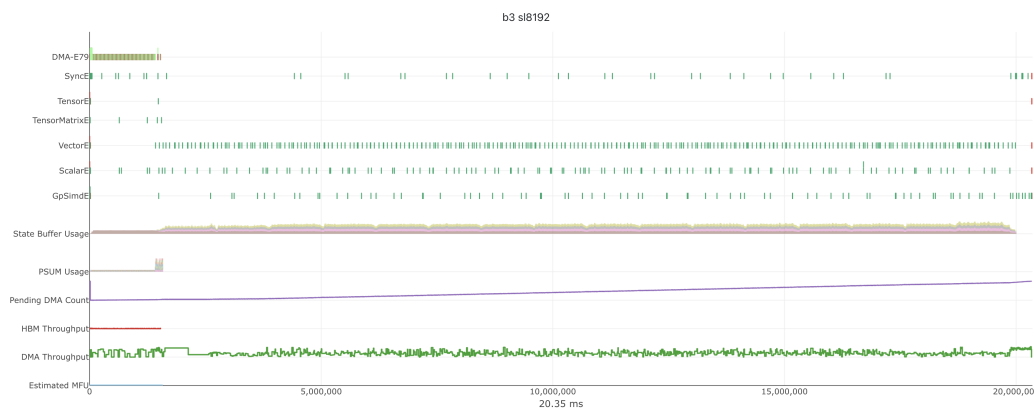


Figure 19: b3, 8k sequence length

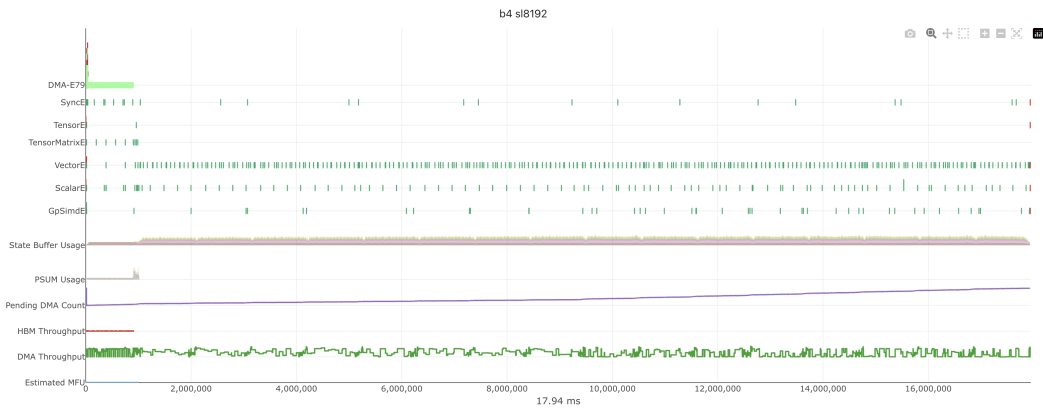


Figure 20: b4, 8k sequence length

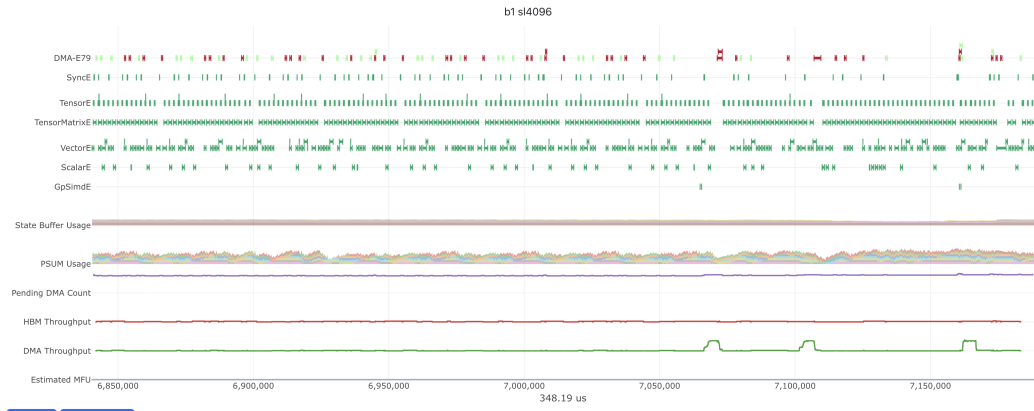


Figure 21: b1, 4k sequence length

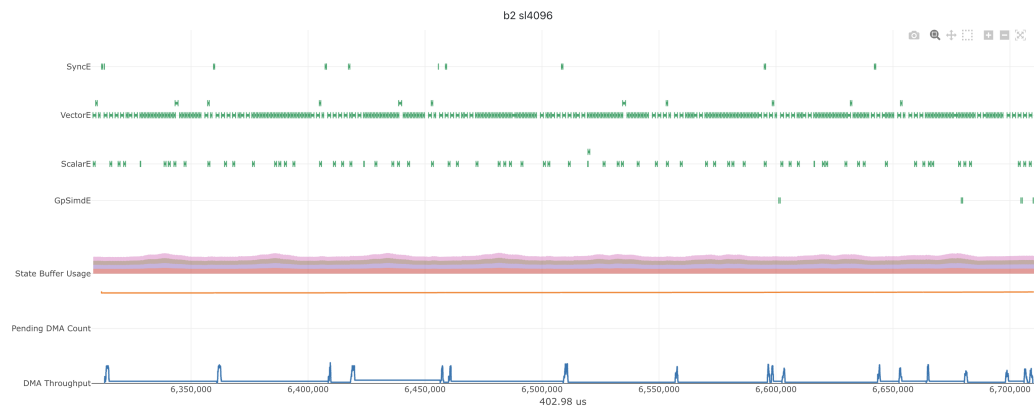


Figure 22: b2, 4k sequence length

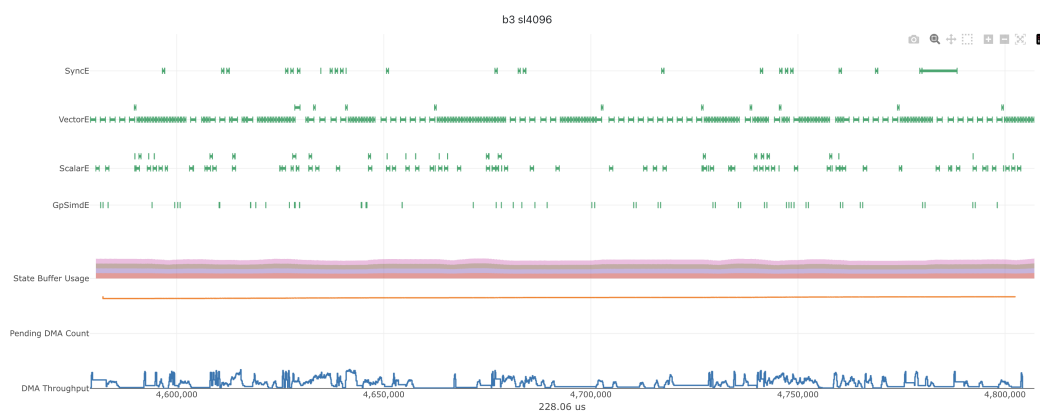


Figure 23: b3, 4k sequence length



Figure 24: b3, 8k sequence length

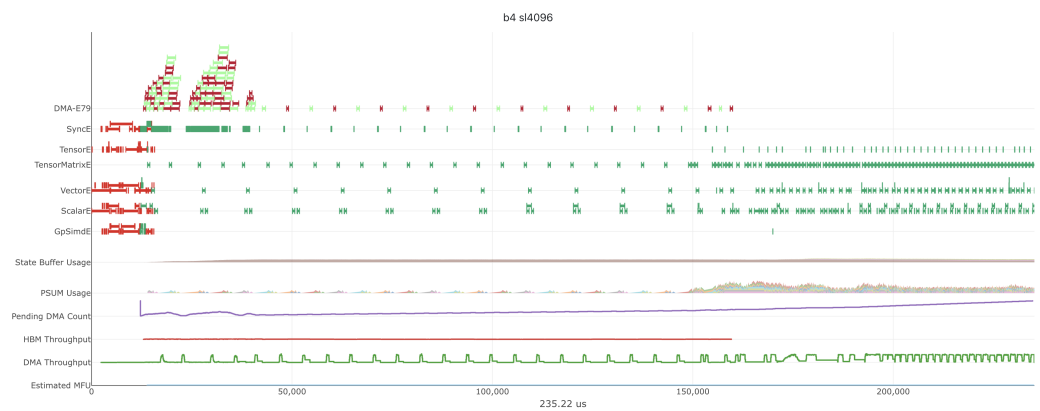


Figure 25: b4, 4k sequence length

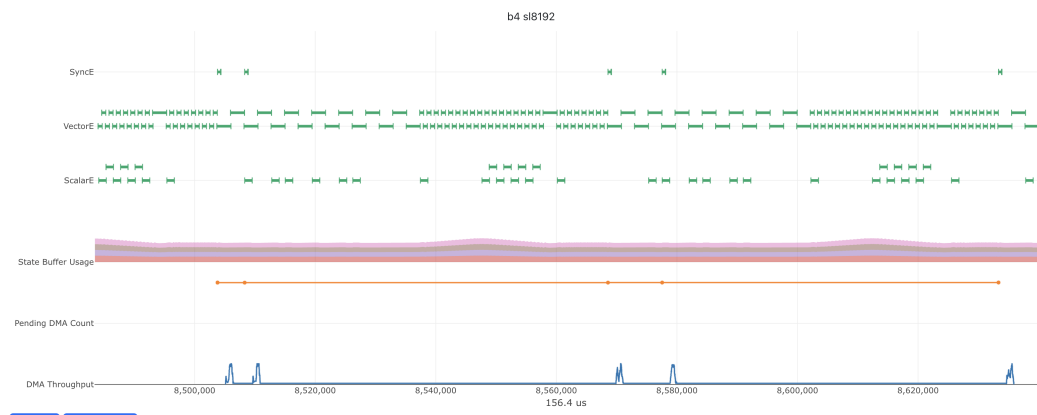


Figure 26: b4, 8k sequence length